

Khalil Smith, Daniel Jaffe, Hien Tran

4/16/2026

Phase 2: Motorcycle Dashboard

Reflection Questions

1. Which shared variables required synchronization and why?

The main shared variable that required synchronization in this program was the `global_state` structure, since it is accessed and modified by multiple threads at the same time. This structure contains important system data such as RPM, engine temperature, speed, fuel level, battery level, ECU status fields, timers, and the event log. Threads like the engine, motion, fuel, ECU, hybrid, event logger, and dashboard all read from and write to these values concurrently. Because of this, synchronization was necessary to prevent race conditions, where threads could overwrite each other's updates or read partially updated data, leading to inconsistent system behavior and incorrect dashboard output.

2. What synchronization mechanisms did you use (mutex, semaphore, condition variable) and how are they implemented in your program?

The program uses mutexes and condition variables to handle synchronization. Mutexes are used to protect critical sections of the code where shared data is accessed or modified, ensuring that only one thread can work with that data at a time. Each subsystem, such as engine, motion, fuel, hybrid, and logging, has its own mutex. Condition variables are used to coordinate thread execution. For example, the motion and fuel threads wait until the engine is turned on using a condition variable, while the ECU thread waits for a signal before processing updated values. The event logger uses a producer-consumer model with condition variables, where events are added to a queue by producer threads and removed by the logger thread, ensuring safe and organized event handling.

3. How did you avoid deadlocks in your implementation?

Deadlocks were avoided by following a consistent lock ordering strategy throughout the program. Whenever multiple locks were needed, they were acquired in the same order and released in reverse order. This prevents circular waiting, which is one of the main causes of deadlocks. Additionally, threads were designed to hold locks for only short periods of time and avoided waiting on condition variables while holding multiple unrelated locks. These practices helped reduce the chances of threads blocking each other indefinitely.

4. Describe the locking strategy used in your program.

The locking strategy in this program is based on per-subsystem locking combined with coordinated access when needed. Each thread typically locks only the subsystem it is responsible for, such as the engine thread using the engine lock or the fuel thread using the fuel lock. This allows different parts of the system to run concurrently without unnecessary blocking. When a thread needs access to multiple subsystems, such as the ECU or dashboard, it locks multiple mutexes in a fixed order. The dashboard uses a snapshot strategy, where it locks all relevant subsystems, copies the global state into a local variable, and then unlocks everything before displaying the data. This ensures that the output is consistent and not affected by concurrent updates.

5. What system rules are enforced by the ECU?

The ECU enforces several important safety and operational rules in the system. It classifies RPM into zones such as IDLE, NORMAL, HIGH, and REDLINE, and temperature into zones such as COLD, NORMAL, HOT, and OVERHEAT. It also monitors fuel levels and identifies low fuel conditions. When the engine is off, the ECU ensures that RPM is set to zero and signals other subsystems accordingly. In cases of overheating, it limits RPM and speed to protect the system, and when fuel is low, it restricts speed to conserve fuel. It also maintains proper idle RPM when the motorcycle is not moving and updates the overall ECU state based on current conditions. Additionally, the ECU logs important events such as entering critical temperature zones or low fuel states.

6. How did synchronization affect the dashboard behavior?

Synchronization improved the dashboard behavior by ensuring that all displayed information is consistent and accurate. The dashboard thread locks all relevant subsystems before copying the

global state into a local snapshot, then unlocks everything before printing. This prevents the dashboard from reading values while they are being updated by other threads. As a result, the dashboard avoids displaying mismatched or partially updated data. Synchronization also ensures that the event log is read safely while it is being updated by the event logger thread, allowing the dashboard to display recent events reliably without corruption or missing information.